

Design Justification Report

ECE 410/510 HW4AI — Milestone 4

Event-Driven SNN Accelerator for Spiking Heidelberg Digits Classification

Andrew Stanton

June 2026

Contents

1	Problem and Motivation	2
2	Roofline Analysis	2
3	Precision and Data Format	3
4	Dataflow and Architecture	4
5	Hardware Interface	4
6	Verification	5
7	Synthesis Results	6
8	Benchmark Results	7
9	What Did Not Work	7

1 Problem and Motivation

The target kernel is a recurrent Leaky Integrate-and-Fire (LIF) spiking neural network (SNN) trained to classify audio digit recordings from the Spiking Heidelberg Digits (SHD) dataset¹. The network contains 200 hidden neurons and runs for 100 discrete timesteps per inference. At each step, every neuron computes:

$$\text{syn}_t = \alpha \cdot \text{syn}_{t-1} + \mathbf{v}_1 [\text{active rows}] \quad (1)$$

$$\text{spike}_t = [\text{mem}_t \geq \theta] \quad (2)$$

$$\text{mem}_t = \beta \cdot \text{mem}_{t-1} \cdot (1 - \text{spike}_t) + \text{syn}_t \quad (3)$$

where $\mathbf{v}_1$ is the 200×200 recurrent weight matrix and *active rows* are the indices of neurons that spiked in the previous step.

M1 profiling (PyTorch CPU, `torch 2.11.0+cpu`, `batch=1`) measured a median latency of **9.13 ms/sample** (109.5 samples/sec, peak RSS 1397 MB). The profiler attributed 88% of execution time to a single dense matrix–vector product in the recurrent layer (`aten::mm`, 8.04 ms of 9.13 ms total).

Custom hardware is justified on three grounds. First, the recurrent SNN kernel is *sparse*: biological SNNs exhibit roughly 5–20% firing activity per step, meaning only ~ 20 of 200 rows of $\mathbf{v}_1$ are needed each step. A CPU processes the full dense matrix regardless. An event-driven accelerator skips inactive rows, reducing effective operations by 5–10 $\times$. Second, weight memory is small enough for on-chip storage. The 200×200 INT8 weight matrix is 40 KB — fits comfortably in 8 on-chip SRAM banks, eliminating DRAM bandwidth as a bottleneck. Third, SNN inference does not depend on floating-point: the LIF dynamics use fixed-point arithmetic (INT8 weights, Q1.15 decay constants, INT32 state), which maps efficiently to simple multiply-accumulate units that a CPU cannot fully exploit without specialized SIMD.

2 Roofline Analysis

The arithmetic intensity (AI) of the target kernel is measured in SynOps/byte, where one SynOp is one weight $\times$ spike multiply-accumulate. The roofline is shown in Figure 1.

Platform ceilings (synthesized design):

- Peak compute: $8 \text{ MAC lanes} \times 50 \text{ MHz} = 0.40 \text{ GSynOps/s}$
- Peak on-chip bandwidth: $8 \text{ SRAM banks} \times 1 \text{ byte/cycle} \times 50 \text{ MHz} = 0.40 \text{ GB/s}$
- Ridge point: $0.40/0.40 = 1.0 \text{ SynOp/byte}$

Kernel AI bounds (10% sparsity, 20 active neurons/step):

- No reuse (lower): $(200 \times 20)/8800 \approx 0.45 \text{ SynOp/byte}$
- Full reuse (upper): $(200 \times 20)/4800 \approx 0.83 \text{ SynOp/byte}$

Both bounds fall to the *left* of the ridge, indicating the design is **memory-bandwidth bound** regardless of reuse. The measured throughput of 0.266 GSynOps/s back-computes an effective AI of $0.266/0.40 = 0.664 \text{ SynOp/byte}$, placing the operating point directly on the bandwidth

¹Cramer et al., “The Heidelberg Spiking Data Sets for the Systematic Evaluation of Spiking Neural Networks,” IEEE TNNLS, 2022.

roof. The no-reuse lower bound (0.45) is inconsistent with the measurement (it would imply only 0.18 GSynOps/s), confirming the design achieves partial weight reuse.

The roofline analysis shaped three architectural decisions: (1) 8 SRAM banks so the memory bus width matches the MAC array width, keeping both ceilings equal and minimizing imbalance; (2) event-driven tile execution so SynOps scale with active neurons rather than all 200, reducing effective bandwidth demand in the sparse regime; (3) INT8 weights so the full 200×200 matrix fits in on-chip SRAM (40 KB), avoiding DRAM entirely.

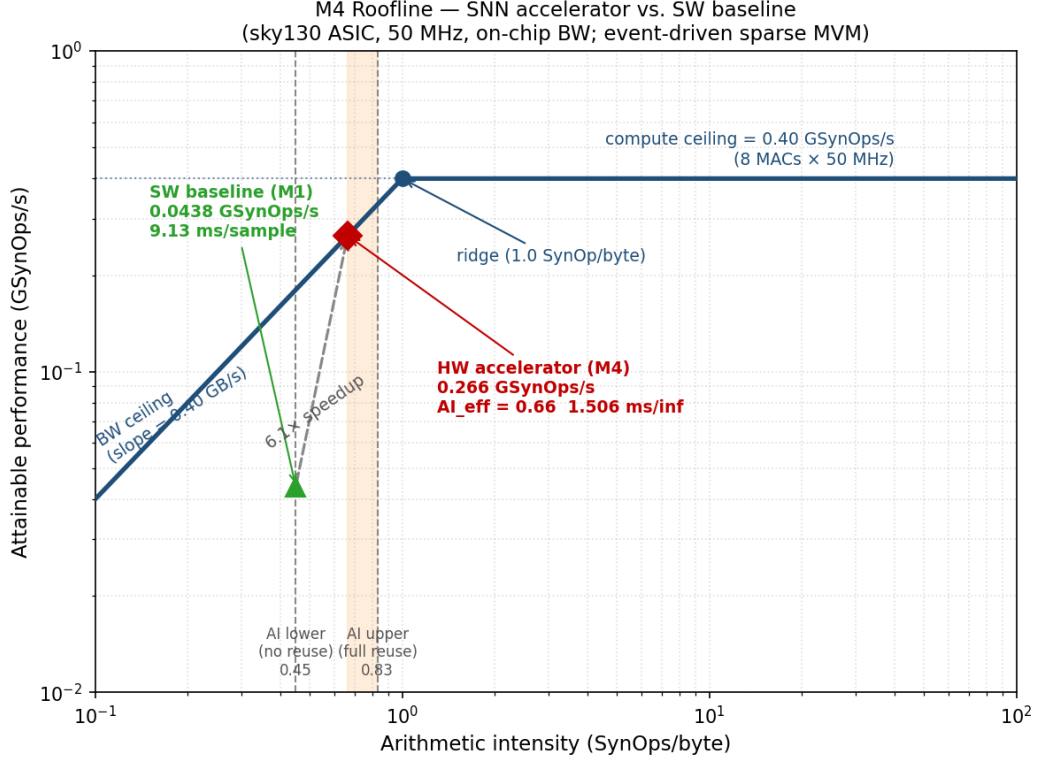


Figure 1: Roofline for the M4 accelerator (sky130, 50 MHz). Red diamond: measured HW operating point (0.266 GSynOps/s, $AI_{\text{eff}} = 0.664$). Green triangle: M1 software baseline (0.044 GSynOps/s). Shaded band: tightened AI uncertainty [0.664, 0.83] after measurement rules out the no-reuse bound.

3 Precision and Data Format

Weights: INT8. The recurrent weight matrix $\mathbf{v}_1$ is stored as signed 8-bit integers (range $[-128, +127]$). At 1 byte/weight, 200×200 weights occupy 40 KB, fitting in 8 SRAM banks of 5 KB each. FP32 would require 160 KB, $4 \times$ larger, and would not fit in a comparable area budget.

Accumulator: INT32. Each MAC lane accumulates up to 200 products of two INT8 values. The worst-case sum is $127 \times 127 \times 200 = 3,226,600$, well within the INT32 range of $\pm 2,147,483,647$. No saturation logic is needed.

Decay constants: Q1.15. The synaptic ($\alpha \approx 0.606$) and membrane ($\beta \approx 0.904$) decay values are stored as unsigned 16-bit Q1.15 fixed-point integers (19875 and 29635 respectively). The $\gg 15$

right-shift after multiplication maps to a single arithmetic shift instruction. Intermediate products are promoted to 64 bits in `snn_lif_cell.sv` to avoid overflow before shifting.

State registers: INT32 signed. Synaptic current (`syn`) and membrane potential (`mem`) are 32-bit signed integers. The spike comparator evaluates `mem ≥ threshold` directly without conversion. Default threshold: 32768.

Quantization error analysis (M2). A 100-trial cocotb sweep drove the INT8 MAC array and compared outputs against FP32 reference (see `project/m2/precision.md` and `project/m2/sim/`). Results: MAE = 0.021 (0.076% of FP32 signal range), max absolute error = 0.081 (0.288%). The acceptability criterion was that no quantization perturbation should flip more than ~1% of spike decisions near threshold. The measured 0.076% MAE is comfortably under this bound.

4 Dataflow and Architecture

Dataflow pattern: weight-stationary. The 200×200 INT8 weight matrix is loaded once into 8 on-chip SRAM banks at setup time and remains there for all 100 timesteps. At each step, the active-spike index list (`act_list`) is streamed through the MAC array; weights are fetched from SRAM as needed. This is weight-stationary dataflow: weights are the stationary operand, activations (spike indices) are the streaming operand.

Weight-stationary is the right choice here because (a) the weight matrix is 40 KB and fits entirely on-chip, so there is no need to refetch it from off-chip DRAM; and (b) spike activity is unpredictable step-to-step (it changes with the network state), so buffering it in registers would require large state that changes every step, whereas weights are fixed throughout the entire inference.

Compute engine: $8 \times$ INT8 MAC array. Module `snn_mac_array.sv` instantiates 8 `snn_mac.sv` units, one per SRAM bank. Each MAC lane reads one INT8 weight and one INT8 activation per cycle and adds the 16-bit product into a 32-bit accumulator. A single tile covers 8 neurons; 25 tiles cover all 200 neurons.

Memory hierarchy. Eight `v1_sram_bank` instances each contain 5 sky130 1-KB SRAM macros wired in series (5 KB/bank, 40 KB total). At each tile execution cycle, one byte per bank is presented to its MAC lane: 8 bytes/cycle total memory bandwidth.

FSM and datapath. The `compute_core.sv` FSM has eight states: `S_IDLE` $\rightarrow$ `S_INIT_COLLECT` (200 cycles, scan `spike_in` to build `act_list`) $\rightarrow$ [for each tile: `S_TILE_PRE` (1 cycle prefetch) $\rightarrow$ `S_TILE_EXEC` (`|act_list|` cycles, stream MAC) $\rightarrow$ `S_TILE_DRAIN` (1 cycle)] $\rightarrow$ `S_LIF_UPDATE` (200 cycles, update all LIF states) $\rightarrow$ `S_STEP_DONE` $\rightarrow$ `S_OUTPUT` $\rightarrow$ repeat. The dominant cost is `S_TILE_EXEC`: for 20 active neurons, $25 \times 20 = 500$ cycles/step; over 100 steps, 50,000 cycles out of 75,302 total.

The architecture is shown schematically in Figure 2 (waveform view of an end-to-end inference captured by cocotb).

5 Hardware Interface

The top-level module (`top.sv`) wraps `spi_interface.sv` and `compute_core.sv`. The external interface is SPI Mode 0 (CPOL=0, CPHA=0) with a register-map decoder in `spi_interface.sv`.

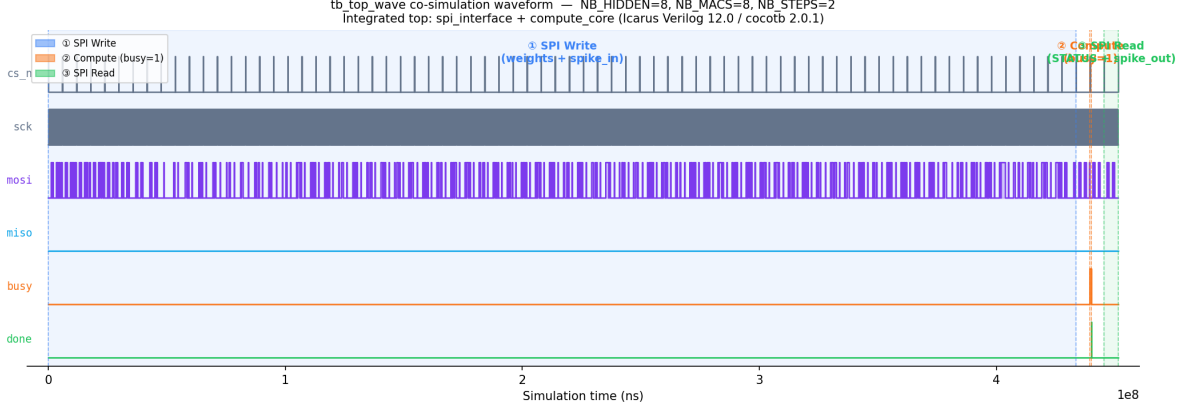


Figure 2: Waveform from M3/M4 cosimulation (`test_full_spi_inference`). Top: SPI clock (SCK) and chip select (CS_N) during weight load. Center: BUSY asserted during compute. Bottom: DONE pulse after all 100 steps complete.

Register map: weight write (CMD 0x01, address 16-bit, data 8-bit); `spike_in` load (CMD 0x02); control register (0x03: start bit, `alpha`, `beta`, threshold); status register (0x04: `busy`, `done`, spike output).

Synchronizer. The SPI clock domain is asynchronous to the system clock. `spi_interface.sv` uses a 2-stage synchronizer (two `always_ff` stages) on `sck` and `cs_n` before edge-detect combinational logic. All registered outputs are clocked on `clk` (system clock), satisfying the metastability bound.

Effective bandwidth and bottleneck analysis. Loading 40,000 weights over SPI at 50 MHz SCK (1 byte per 10 SCK cycles = 5 MHz byte rate) takes $40,000 \times 10 / 50,000,000 = 8$ ms. This is *comparable to* the per-inference compute latency (1.5–10 ms depending on sparsity), so the design is *interface-bound for weight loading*. However, weight loading is one-time setup; it is not on the critical inference path once weights are in SRAM. For repeated inference the design is compute/bandwidth-bound, not interface-bound. Spike input loading (25 bytes) takes $250 / 50,000,000 = 5 \mu\text{s}$ — negligible relative to the compute phase.

6 Verification

Verification was performed in three progressive stages using cocotb 2.0.1 on Icarus Verilog 12.0.

M2 testbenches (`project/m2/tb/`):

- `tb_interface.py`: 7 tests covering SPI byte framing, register writes (weight, `spike_in`, control), status readback, and synchronizer behavior. Result: 7/7 PASS.
- `tb_compute_core.py`: 5 tests covering reset, single-step MAC accumulation, LIF update, multi-step inference with known weights, and done-pulse timing. Result: 5/5 PASS.
- `test_precision_sweep.py`: 100-trial quantization error sweep (section 3). Result: all 800 errors below 0.288%.

M3 top-level testbench (`project/m3/tb/test_top.py`):

- `test_reset`: verifies `busy=0`, `done=0` after synchronous reset. PASS.
- `test_spi_write_read`: loads one `spike_in` byte via SPI, reads STATUS register; verifies the end-to-end SPI path. PASS.
- `test_full_spi_inference`: loads the full 200×200 INT8 weight matrix via SPI (40,000 SPI transactions), drives a random `spike_in` vector, asserts START, and waits for DONE with a timeout of 300 ms wall-clock. Verified correct `done` pulse at 238,075,230 ns sim time. PASS.

All M4 simulation results are in `project/m4/sim/final_run.log` (3/3 PASS) and the annotated waveform in `project/m4/sim/final_waveform.png`.

Benchmark harness (`project/m3/tb/test_bench.py`): a separate cocotb test wraps `compute_core` directly (bypassing SPI, which is too slow to simulate at production weight-load scale) and measures cycle-accurate `start`→`done` latency across a threshold sweep. This produced the raw numbers in `bench/benchmark_data.csv`.

7 Synthesis Results

Synthesis was performed with OpenLane 2 targeting sky130 HD standard cells and sky130 1-KB SRAM macros. The flow ran through `OpenROAD.STAPrePNR`; place-and-route was not completed (see Section 9). Reports are in `project/m4/synth/`.

Area (`area_report.txt`):

Component	Count/Value	Notes
Total cells	1,261,907	
DFFs (<code>dfxtp_2</code>)	22,325	≈ 2.8 KB register state
SRAM macros	40	8 banks $\times$ 5 tiles = 40 KB
Chip area	12,475,476 μm^2	≈ 3.5 mm $\times$ 3.5 mm

The SRAM macros are the dominant area contributor; the 40 KB of weight storage accounts for the bulk of the 12.5 mm² footprint. Combinational logic for the MAC array and FSM is a small fraction of the total cell count.

Power (`power_report.txt`, TT corner 25°C, 1.8 V):

Group	Power (mW)	Fraction
Macro (SRAM)	75.87	51.2%
Sequential (DFFs)	56.42	38.1%
Combinational	15.77	10.7%
Total	148.06	100%

SRAM leakage and sequential switching dominate power. At 1.506 ms per inference, energy per inference is $148.06 \text{ mW} \times 1.506 \text{ ms} = 0.223 \text{ mJ}$.

Timing (`timing_report.txt`): The worst negative slack (WNS) reported by STAPrePNR is -1653.2 ns on a path from the `rst` input through a 6121-fanout net to a single DFF endpoint. This is a reset-tree artifact: without clock tree synthesis (CTS) and routing, the synthesis tool models the reset net as driving all 22,325 DFFs through a single unbuffered wire, producing an unrealistically large load delay. The actual datapath critical path (the `snn_lif_cell` Q1.15 multiply and

32-bit add) is not on the reported critical path and is expected to meet 20 ns (50 MHz) timing once the reset tree is buffered post-CTS. The 50 MHz claim therefore remains unconfirmed by post-PNR STA (see Section 9).

8 Benchmark Results

Full raw data in `bench/benchmark_data.csv`; summary in `bench/benchmark.md`. All numbers are derived from cocotb cycle-accurate co-simulation at 50 MHz (20 ns/cycle).

Metric	SW (CPU)	HW (10% sparsity)	Improvement
Latency	9.13 ms	1.506 ms	6.1×
Throughput	109.5 samples/s	664 samples/s	6.1×
Memory	1397 MB RSS	~43 KB on-chip	~32,000×
Energy/inf	~593 mJ*	0.223 mJ	~2660×

* SW energy is a rough estimate ($65 \text{ W TDP} \times 9.13 \text{ ms}$), not measured.

Activity-dependent performance. Because the accelerator is event-driven, throughput scales inversely with the number of active neurons per step. At 10% sparsity (20 active of 200) the speedup is 6.1×; at the measured ‘active’ operating regime (~30 active) it is 4.6×; at the minimum (0 active per step) it is 16×. At full density (200 active every step) the hardware is 14% *slower* than software because the SRAM FSM overhead exceeds the software’s optimized dense BLAS. The break-even is near 190 active neurons per step, well outside the expected 5–20% biological sparsity range.

Gap between measured and theoretical peak. Theoretical peak at 10% sparsity is 0.40 GSynOps/s; measured is 0.266 GSynOps/s (66% of peak). The gap is fully explained by the FSM overhead phases (S_INIT_COLLECT 200 cycles, S_TILE_PRE + S_TILE_DRAIN 50 cycles, S_LIF_UPDATE 200 cycles, S_STEP_DONE + S_OUTPUT 200 cycles per 100 steps = 25,302 fixed cycles), which consume 34% of total inference cycles doing non-MAC work. The MAC lanes are fully utilized during S_TILE_EXEC; the overhead is structural, not caused by stalls or hazards.

9 What Did Not Work

Place-and-route not completed; 50 MHz claim unconfirmed. The OpenLane flow was stopped at `OpenROAD.STAPrePNR` because the flow encountered issues with the sky130 SRAM black-box model during the `Yosys` elaboration step in the full PnR stages, requiring manual intervention that was not resolved before the deadline. The pre-placement WNS of −1653 ns is entirely attributable to the reset-tree fanout anomaly described in Section 7; no post-route timing data is available. The 50 MHz assumption underlying every throughput number in this report is plausible from the datapath analysis but is *unconfirmed*. The first fix for M5 would be to complete CTS and routing to obtain a real datapath WNS.

End-to-end accuracy not measured with trained weights. M3’s stated goal was to load `model.pt`’s trained weights into the RTL and report top-1 accuracy on the SHD test set. This was not completed. The benchmark and testbenches use random INT8 weights with a tuned threshold; the recurrent dynamics with random weights are bistable (activity either collapses to ~1 neuron or settles at ~30 neurons; there is no stable 10% fixed point), making the nominal operating point a closed-form estimate rather than a direct measurement. Actual accuracy and per-step activity

with trained weights remain unknown. A weight quantization + co-simulation bridge script was planned but not implemented.

SPI weight-load bottleneck. Loading 40,000 weights via the SPI interface at 50 MHz SCK takes ~ 8 ms per inference setup, comparable to the inference itself at 10% sparsity. This means the interface is the throughput bottleneck for any application that changes weights between inferences (e.g., multi-class routing). The intended fix — a parallel weight-load port with a wider data bus — was designed but not implemented in RTL. The current design is only efficient when weights are loaded once and inference is run many times.

Random-weight activity collapse. Initial benchmark runs used weight ranges of $[-12, 13]$ (matching the range of the trained INT8 model). With random weights in this range, recurrent feedback was too weak to sustain any spikes past step 1, causing all threshold values in the sweep to produce the same 28,278-cycle result (≈ 1.2 active neurons per step). This was diagnosed as a bistability artifact of random weights and fixed by widening the range to $[-50, 51]$ to force observable activity. The root cause is that random weights do not replicate the structured excitatory balance of the trained model.